



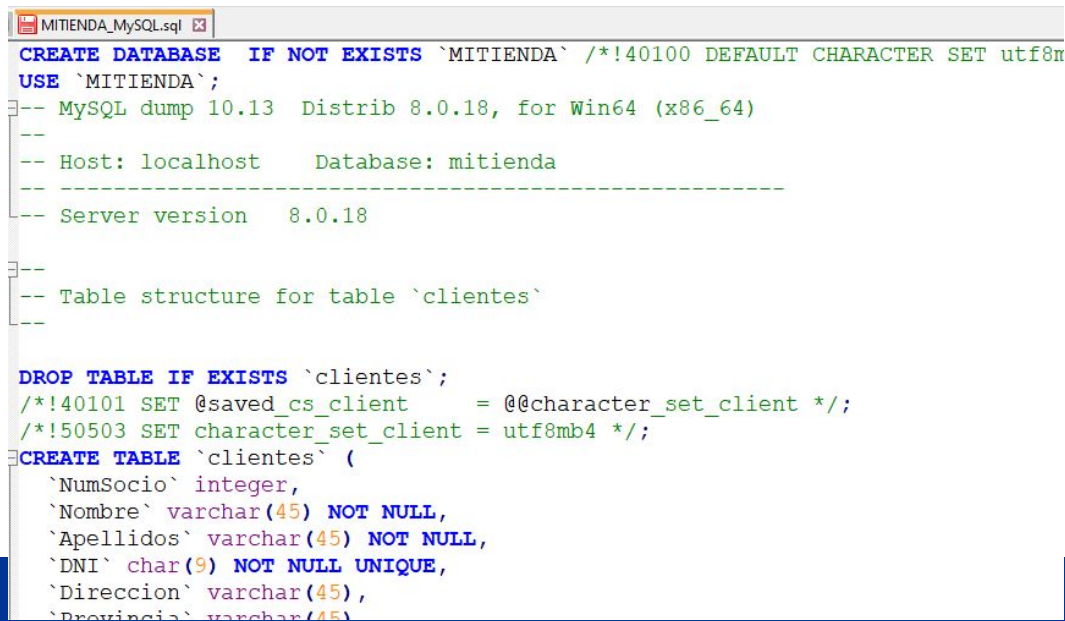
## Tema 7. Automatización de la BD

Σavante

# Guiones en MySQL

Un guión (en inglés, *script*) es un **conjunto de instrucciones ordenadas** que se **ejecutan secuencialmente**.

El guión se escribe en un **fichero de texto** con extensión **.sql**.

A screenshot of a text editor window showing a MySQL script. The title bar reads 'MITIENDA\_MySQL.sql'. The script contains SQL commands to create a database, drop a table, and create a table named 'clientes'. It also includes comments about the MySQL version and table structure.

```
MITIENDA_MySQL.sql
CREATE DATABASE IF NOT EXISTS `MITIENDA` /*!40100 DEFAULT CHARACTER SET utf8m
USE `MITIENDA`;
-- MySQL dump 10.13  Distrib 8.0.18, for Win64 (x86_64)
--
-- Host: localhost    Database: mitienda
--
-- Server version    8.0.18
--
-- Table structure for table `clientes`
--
DROP TABLE IF EXISTS `clientes`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `clientes` (
  `NumSocio` integer,
  `Nombre` varchar(45) NOT NULL,
  `Apellidos` varchar(45) NOT NULL,
  `DNI` char(9) NOT NULL UNIQUE,
  `Direccion` varchar(45),
  `Provincia` varchar(45)
```



---

# Rutinas en MySQL

## Procedimientos, funciones, parámetros y transacciones

Una rutina es un **conjunto de instrucciones SQL** al que se le **asigna un nombre** y se almacena en el servidor.

Para ejecutar estas instrucciones sólo hay que llamar a la rutina con el nombre asignado.

Las rutinas que se pueden crear en MySQL son:

- **Procedimientos** almacenados
- **Funciones**
- Disparadores/**Triggers**
- **Eventos**



# Rutinas en MySQL

## Procedimientos almacenados

```
CREATE PROCEDURE <nombre_procedimiento> ([<lista_parámetros>])  
BEGIN  
    <lista_sentencias_SQL>  
END
```

Los parámetros pueden ser de 3 tipos:

- **IN**: entrada de información al procedimiento
- **OUT**: salida de información del procedimiento
- **INOUT**: entrada-salida



# Rutinas en MySQL

## Procedimientos almacenados

→ El formato de cada parámetro del procedimiento es:

`[IN | OUT | INOUT] <nombre_parámetro> <tipo_datos>`

→ Si no se indica el tipo de parámetro, por defecto será **IN**.

→ En el caso de no haber parámetros, se ponen sólo los paréntesis **()**

→ Para **lanzar** un procedimiento, se hace con la sentencia **CALL**:

```
CALL <nombre_procedimiento> ([<lista_parámetros>]);
```



---

# Rutinas en MySQL

## Cláusula DELIMITER

La palabra clave **DELIMITER** indica el carácter de comienzo y fin de la rutina. Se suelen usar los caracteres **\$\$**, **&&** o **//**

Es importante poner esta cláusula siempre antes de comenzar el código de **cualquier rutina**, ya que si no MySQL interpretará el comando sólo hasta el primer ;



# Rutinas en MySQL

## Procedimientos almacenados (Ejemplos)

```
1 DELIMITER $$
2 • CREATE PROCEDURE P_SALUDAR ()
3 BEGIN
4     SELECT "Hola mundo!";
5 END $$
6
7 • CALL P_SALUDAR();
```

<

Result Grid | Filter Rows:  | Export: | Wrap Cell Content:

|   |             |
|---|-------------|
|   | Hola mundo! |
| ▶ | Hola mundo! |



# Rutinas en MySQL

## Procedimientos almacenados (Ejemplos)

```
1  DELIMITER $$
2 • CREATE PROCEDURE P_SALUDAR_NOMBRE (IN x VARCHAR(20))
3  BEGIN
4      SELECT CONCAT("Hola ",x);
5  END $$
6
7 • CALL P_SALUDAR_NOMBRE ("Rocío");
```

Result Grid |  Filter Rows:  | Export:  | Wrap Cell Content: 

|                   |
|-------------------|
| CONCAT("Hola ",x) |
| Hola Rocío        |





# Rutinas en MySQL

## Procedimientos almacenados (Ejemplos)

```
1 DELIMITER $$
2 • CREATE PROCEDURE p_devolver_saludo (OUT x VARCHAR(30))
3 BEGIN
4     SET x = "Hola mundo";
5 END $$
6
7 • CALL p_devolver_saludo (@mivariable);
8 SELECT @mivariable;
```

Result Grid | Filter Rows:  | Export: | Wrap Cell Contents:

|   | @mivariable |
|---|-------------|
| ▶ | Hola mundo  |



# Rutinas en MySQL

## Procedimientos almacenados (Ejemplos)

```
1 DELIMITER $$
2 • CREATE PROCEDURE p_contador (INOUT x INTEGER)
3   BEGIN
4       SET x = x + 1;
5   END $$
6
7 • SET @mivariablecontador = 3;
8   CALL p_contador (@mivariablecontador);
9   SELECT @mivariablecontador;
```

< Result Grid | | Filter Rows:  | Export: | Wrap

|   | @mivariablecontador |
|---|---------------------|
| ▶ | 4                   |



# Rutinas en MySQL

## Funciones

```
CREATE FUNCTION <nombre_función> ([<lista_parámetros>])  
  RETURNS <tipo_valor_devuelto>  
  [NOT] DETERMINISTIC  
  BEGIN  
    <lista_sentencias_SQL>  
    RETURN <valor>;  
  END
```

Las funciones **devuelven SIEMPRE un valor** (sólo uno). Hay que indicar el tipo en la cláusula **RETURNS**. El valor que devuelve la función se asigna con **RETURN**.



# Rutinas en MySQL

## Funciones

- Los parámetros de las funciones son siempre de tipo IN, y **no hace falta indicarlo**. Por tanto, el formato de cada parámetro será:

<nombre\_parámetro> <tipo\_datos>

- En el caso de no haber parámetros, se ponen sólo los paréntesis ()
- La cláusula **DETERMINISTIC** indica que la función devuelve siempre el mismo resultado cuando se lanza con los mismos parámetros.
- Para **lanzar** una función, se hace dentro de una sentencia **SELECT**:

```
SELECT <nombre_función> ([<lista_parámetros>]);
```



# Rutinas en MySQL

## Funciones (Ejemplo)

```
1  DELIMITER $$
2 • CREATE FUNCTION F_SALUDAR ()
3  RETURNS CHAR(11)
4  DETERMINISTIC
5  BEGIN
6      RETURN "Hola mundo!";
7  END $$
8
9 • SELECT F_SALUDAR ();
```



The screenshot shows a MySQL IDE interface. At the top, a code editor contains SQL code for creating a function and calling it. Below the editor is a toolbar with icons for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The 'Result Grid' is active, displaying a single row with the output of the function call.

| F_SALUDAR () |
|--------------|
| Hola mundo!  |



---

# Rutinas en MySQL

## Variables de usuario

Para algunas rutinas el usuario necesitará crear **variables**.

Una variable es un **espacio de memoria reservado** en el que se puede escribir un valor para después consultarlo.

No hace falta crear explícitamente la variable, al inicializarla o utilizarla en alguna llamada a un procedimiento **se crea automáticamente**.



---

# Rutinas en MySQL

## Variables de usuario (operaciones)

- **Creación de variables:** No hace falta crear explícitamente la variable, al inicializarla o utilizarla se crea automáticamente.
- **Nombramiento:** Las variables se nombran como **@nombre\_variable**
- **Consulta:** para consultar el valor de una variable, se hace dentro de una sentencia SELECT:

```
SELECT @nombre_variable;
```



# Rutinas en MySQL

## Variables de usuario (operaciones)

- **Asignar valor:** Para asignar valor a una variable, lo podemos hacer de 2 maneras:

- Para un valor concreto: `SET @nombre_variable = <valor>;`
- Para un valor devuelto por una consulta:

```
SELECT <valor>  
  
INTO @nombre_variable  
  
FROM <tabla>;
```





# Rutinas en MySQL

## Instrucciones condicionales

| SIMPLES                                | DOBLES                                                                                          | MÚLTIPLES                                                                                                          |
|----------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| IF-THEN                                | IF-THEN-ELSE                                                                                    | CASE                                                                                                               |
| IF expr1 THEN<br>sentencias<br>END IF; | IF expr1 THEN<br>sentencias<br>ELSEIF expr2 THEN<br>sentencias<br>ELSE<br>sentencias<br>END IF; | CASE expr<br>WHEN valor THEN<br>sentencias<br>[WHEN valor THEN<br>sentencias]<br>[ELSE<br>sentencias]<br>END CASE; |



# Rutinas en MySQL

## Instrucciones repetitivas

| LOOP                                                         | REPEAT UNTIL                                                                    | WHILE LOOP                                                                  |
|--------------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| [etiqueta:] LOOP<br>instrucciones<br>END LOOP<br>[etiqueta]; | [etiqueta:] REPEAT<br>Instrucciones<br>UNTIL expresión<br>END REPEAT [etiqueta] | [etiqueta<br>:] WHILE expresión DO<br>instrucciones<br>END WHILE [etiqueta] |



---

# Rutinas en MySQL

## Cursores

A veces, dentro del código de un procedimiento, nos interesa recuperar varias filas de una tabla y luego **procesarlas una a una**.

Para ello, existe un tipo de variable llamada **CURSOR**.

Los cursores almacena el resultado de una consulta SELECT con varias columnas y filas, y luego se pueden ir recorriendo utilizando varias instrucciones:

- ★ Declaración del cursor: **DECLARE**
- ★ Apertura del cursor: **OPEN**
- ★ Recogida de información (recorrer filas): **FETCH**



# Rutinas en MySQL

## *Handlers/manejadores de errores*

A veces, durante la ejecución de un procedimiento, se puede producir un error (error en tiempo de ejecución). Por ejemplo: intentar insertar una fila con una PK duplicada, intentar acceder a un campo que no existe en una tabla...

Para ello, existe un control llamado **HANDLER** o **manejador de excepciones**.

Se ejecuta cuando se produce el error para el que está programado.

```
DECLARE {CONTINUE | EXIT} HANDLER FOR <cod_error> [, <cod_error>]...  
BEGIN  
    [<lista_instrucciones>]  
END
```



---

# Rutinas en MySQL

## Triggers (disparadores)

Son procedimientos que se ejecutan cuando se intentan modificar los datos de una tabla.

Un trigger se crea para **una tabla específica** y se asocia a una **instrucción DML** sobre dicha tabla.

Ejemplo: un trigger que, después de insertar un nuevo alumno en la tabla ALUMNOS, sume 1 al número de alumnos totales de nuestra BD (que puede estar recogido en otra tabla).



---

# Rutinas en MySQL

## Triggers (disparadores)

```
CREATE TRIGGER <nombre_trigger>
[BEFORE|AFTER] [INSERT|DELETE|UPDATE]
ON <nombre_tabla>
FOR EACH ROW
BEGIN
    <lista_instrucciones>;
END
```



---

# Rutinas en MySQL

## Triggers (disparadores)

Si queremos acceder a la información de la fila que se está procesando durante la sentencia que ha disparado el trigger, disponemos de los registros **OLD** y **NEW**.

Para referirnos a un campo del registro antiguo, debemos poner **OLD.<nombre\_campo>**

Para referirnos a un campo del registro nuevo, debemos poner **NEW.<nombre\_campo>**



---

# Rutinas en MySQL

## Triggers (disparadores)

Valores de los registros **OLD** y **NEW** en un trigger:

|               | <b>OLD</b>                                                | <b>NEW</b>                                                             |
|---------------|-----------------------------------------------------------|------------------------------------------------------------------------|
| <b>INSERT</b> | No definido; todos los campos toman valor NULL.           | Valores que se insertan cuando se complete la inserción de la fila.    |
| <b>UPDATE</b> | Valores originales de la fila, antes de la actualización. | Nuevos valores que se registrarán cuando se complete la actualización. |
| <b>DELETE</b> | Valores originales, antes del borrado de la fila.         | No definido; todos los campos toman valor NULL.                        |





---

# Rutinas en MySQL

## Eventos

Son tareas que se ejecutan de acuerdo a un horario.

Hay eventos para sólo una ocasión o eventos periódicos.

Para habilitar el programador de eventos, se debe activar la variable global *event\_scheduler*:

```
SET GLOBAL event_scheduler = ON;
```



# Rutinas en MySQL

## Eventos

```
CREATE EVENT <nombre_evento>  
ON SCHEDULE {AT <fecha_y_hora> | EVERY <X medida_tiempo>  
[STARTS <fecha_y_hora>] [ENDS <fecha_y_hora>]  
[ON COMPLETION [NOT] PRESERVE]  
DO <lista_instrucciones>;
```

En lugar de <fecha\_y\_hora> se puede poner:

```
TIMESTAMP [+ INTERVAL <X medida_tiempo>]
```



**Σavante**